

# CS & IT ENGINEERING



## Operating System

Deadlock

Lecture -3



By- Vishvadeep Gothi sir

# Recap of Previous Lecture



**Topic**

**Deadlock**

**Topic**

**Deadlock Avoidance**

**Topic**

**Banker's Algorithm**



# Topics to be Covered



**Topic**

**Banker's Algorithm**

**Topic**

**Deadlock Detection**

safe state

but 2 processes  $P_i$  and  $P_j$  requested for  $Req_i$  and  $Req_j$

1. only  $Req_i$  granted
2. —||—  $Req_j$  —||—
3. Neither  $Req_i$  or  $Req_j$  granted
4. Both  $Req_i$  and  $Req_j$  granted
5. Either  $Req_i$  or  $Req_j$  granted but not both together



## Topic : Banker's Algorithm

| Process        | Allocation                | Max   | Available                     | Need                      |
|----------------|---------------------------|-------|-------------------------------|---------------------------|
|                | A B C                     | A B C | A B C                         | A B C                     |
| P <sub>0</sub> | 0 1 0                     | 7 5 3 | <del>3 3 2</del>              | 7 4 3 ✓                   |
| P <sub>1</sub> | <del>3 0 2</del><br>2 0 0 | 3 2 2 | <del>2 3 0</del><br>2 2 0     | <del>0 2 0</del><br>1 2 2 |
| P <sub>2</sub> | 3 0 2                     | 9 0 2 | After P <sub>1</sub><br>5 2 2 | 6 0 0 ✓                   |
| P <sub>3</sub> | <del>2 2 1</del><br>2 1 1 | 2 2 2 | After P <sub>3</sub><br>7 4 3 | <del>0 0 1</del><br>0 1 1 |
| P <sub>4</sub> | 0 0 2                     | 4 3 3 | .                             | 4 3 1 ✓                   |



#Q. What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?  $Req_1 = \langle 1, 0, 2 \rangle$   
What will happen if process P3 requests one additional instance of resource type B?  $Req_3 = \langle 0, 1, 0 \rangle$

Ans:- Both requests are granted together



## Topic : Banker's Algorithm

| Process        | Allocation | Max   | Available | Need  |
|----------------|------------|-------|-----------|-------|
|                | A B C      | A B C | A B C     | A B C |
| P <sub>0</sub> | 0 1 0      | 7 5 3 | 3 3 2     | 7 4 3 |
| P <sub>1</sub> | 2 0 0      | 3 2 2 |           | 1 2 2 |
| P <sub>2</sub> | 3 0 2      | 9 0 2 |           | 6 0 0 |
| P <sub>3</sub> | 2 1 1      | 2 2 2 |           | 0 1 1 |
| P <sub>4</sub> | 0 0 2      | 4 3 3 |           | 4 3 1 |

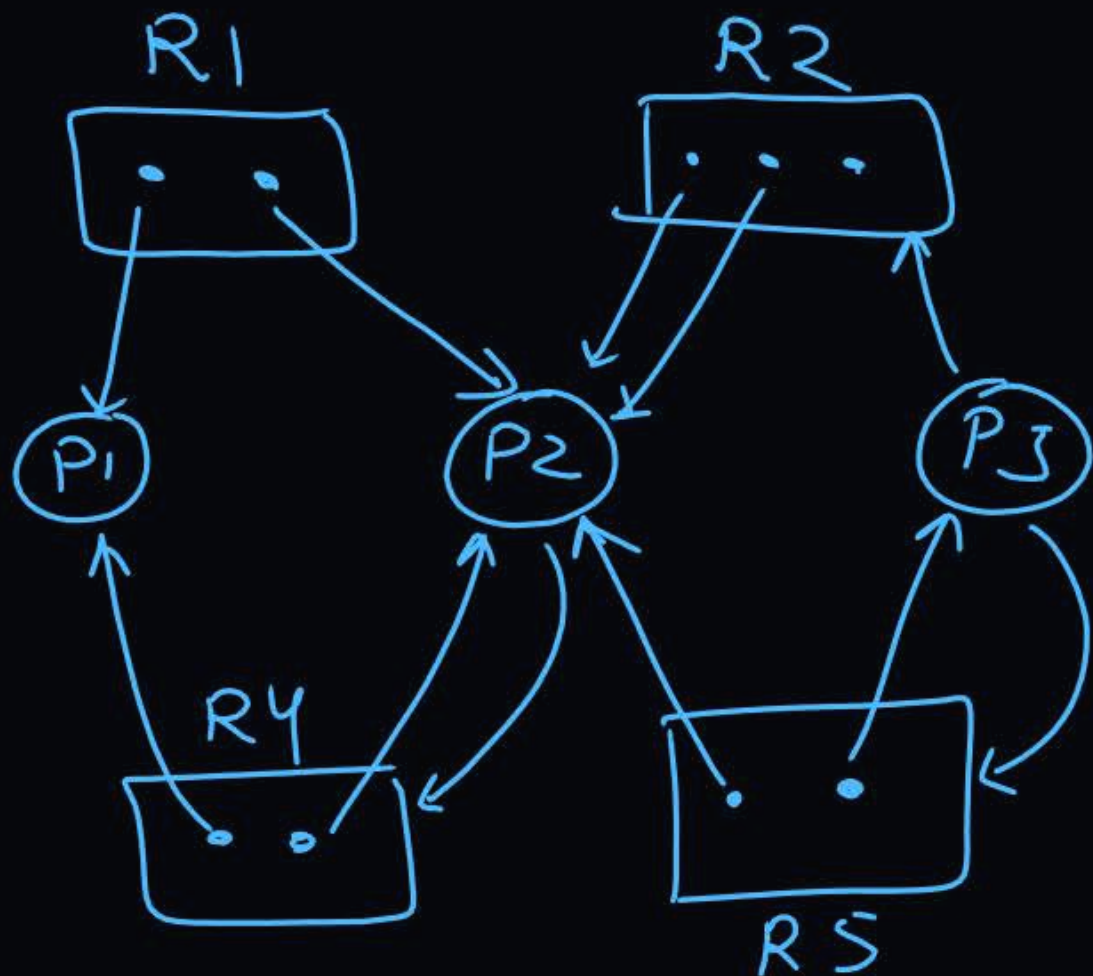
#Q. What will happen if process P0 requests one additional instance of resource type C ?

What will happen if process P3 requests one additional instance of resource type B and one instance of resource type C?





Ques)



Given a resource allocation graph. Check deadlock or not?

|    | Allocation |    |    |    | Request |    |    |    |
|----|------------|----|----|----|---------|----|----|----|
|    | R1         | R2 | R4 | R5 | R1      | R2 | R4 | R5 |
| P1 | 1          | 0  | 1  | 0  | 0       | 0  | 0  | 0  |
| P2 | 1          | 2  | 1  | 1  | 0       | 0  | 1  | 0  |
| P3 | 0          | 0  | 0  | 1  | 0       | 1  | 0  | 1  |

|          | Available |    |    |    |               |
|----------|-----------|----|----|----|---------------|
|          | R1        | R2 | R4 | R5 |               |
|          | 0         | 1  | 0  | 0  | } no deadlock |
| After P1 | 1         | 1  | 1  | 0  |               |
| After P2 | 2         | 3  | 2  | 1  |               |
| After P3 | 2         | 3  | 2  | 2  |               |

Ans



## Topic : Detection-Algorithm Usage

When should the deadlock detection be done? Frequently, or infrequently?





## Topic : Detection-Algorithm Usage

1. Do deadlock detection after every resource allocation
2. Do deadlock detection only when there is some clue

↓  
lower CPU utilization





## Topic : Recovery From Deadlock



There are three basic approaches to recovery from deadlock:

1. Inform the system operator and allow him/her to take manual intervention
2. Terminate one or more processes involved in the deadlock
3. Preempt resources.





## Topic : Process Termination

Terminate all processes involved in the deadlock

Terminate processes one by one until the deadlock is broken





## Topic : Resource Preemption

Important issues to be addressed when preempting resources to relieve deadlock:

1. Selecting a victim  $\Rightarrow$  from which process which resources should be preempted.
2. Rollback  $\Rightarrow$
3. Starvation  $\Rightarrow$  The victim process may go into starvation



#Q. Consider a system with 3 processes A, B and C. All 3 processes require 4 resources each to execute. The minimum number of resources the system should have such that deadlock can never occur?  $\Rightarrow \underline{\underline{10}}$

|   |   | Allocation | Available |
|---|---|------------|-----------|
| A | 4 | 3          | <u>1</u>  |
| B | 4 | 3          |           |
| C | 4 | 3          |           |
|   |   | (max - 1)  |           |

$$3 + 3 + 3 + 1 = 10$$

with max 9 resources also there is a possibility of deadlock

#Q. Consider a system with 4 processes A, B, C and D. All 4 processes require 6 resources each to execute. The maximum number of resources the system should have such that deadlock may occur?

| allocation |   |       |
|------------|---|-------|
| A          | 6 | 5     |
| B          | 6 | 5     |
| C          | 6 | 5     |
| D          | 6 | 5     |
|            |   | <hr/> |
|            |   | 20    |
|            |   | <hr/> |

Ans = 20

for deadlock may occur

$$\sum_{i=1}^n (\max x_i - 1)$$

for no deadlock ever

$$\left[ \sum_{i=1}^n (\max x_i - 1) \right] + 1$$



| <u>Ques)</u> | max | allocation | <u>available</u><br>1 |
|--------------|-----|------------|-----------------------|
| A            | 5   | 4          |                       |
| B            | 6   | 5          |                       |
| C            | 2   | 1          |                       |
| D            | 3   | 2          |                       |

$$4 + 5 + 1 + 2 + 1 = 13$$

min resources for which no deadlock ever = 13 }

Ques)  $n$  processes each require 4 resources.

Total 24 resources are there.

max value of  $n$  for which no any deadlock ever?

Sol<sup>n</sup>

$$n * (4 - 1) + 1 \leq 24$$

$$3n \leq 23$$

$$n \leq 7.66$$

$$\boxed{n_{\max} < 7} \text{ Ans.}$$

#Q. Consider a system with 3 processes that share 4 instances of the same resource type. Each process can request a maximum of K instances. Resource instances can be requested and released only one at a time. The largest value of K that will always avoid deadlock is 2.

$$3(k-1) + 1 \leq 4$$

$$3k - 3 \leq 3$$

$$3k \leq 6$$

$$k \leq 2$$

$$k_{\max} = 2$$



Ques)                      max allocation

|   |   |   |   |
|---|---|---|---|
| A | 5 | 2 | 2 |
| B | 2 | 0 | 1 |
| C | 4 | 1 | 2 |

+ 1 = 6

min available resources  
to never<sup>v</sup> deadlock 6 ?  
have



## Topic : Types of Locks

1. Spinlock <sup>(busy waiting)</sup> → one process running a forever loop `while(True);`
2. Livelock ⇒ A deadlock only but the processes will be in Ready ↔ Running states
3. Deadlock ⇒ when processes are blocked

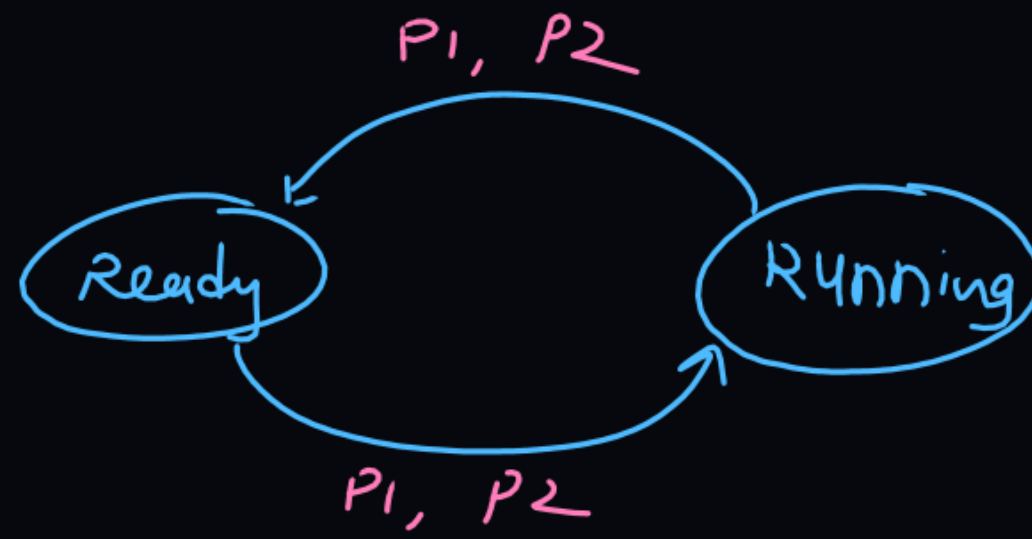
4. ~~Semaphores~~

5. Reentrant Locks

⇓  
one process can lock  
same item multiple times



P1, P2





## Solution 1

*Boolean lock=false;*

```
while(true)  
{  
    while(lock);  
    lock=true;  
    //CS  
    lock=false;  
    RS;  
}
```

```
while(true)  
{  
    while(lock);  
    lock=true;  
    //CS  
    lock=false;  
    RS;  
}
```

## Solution 2

```
int turn=0;
```

```
while(true)
{
    while(turn!=0);
    CS
    turn=1;
    RS;
}
```

```
while(true)
{
    while(turn!=1);
    CS
    turn=0;
    RS;
}
```

## Peterson's Solution

```
Boolean Flag[2]={False, False};  
int turn;
```

```
while(true) {  
    Flag[0]=true;  
    turn=1;  
    while(Flag[1] && turn==1);  
        CS  
    Flag[0]=False;  
    RS;  
}
```

```
while(true){  
    Flag[1]=true;  
    turn=0;  
    while(Flag[0] && turn==0);  
        CS  
    Flag[1]=False;  
    RS;  
}
```



## Question 1

```
turn=0;
```

```
while(true)
{
    while(turn);
    turn=1;
    //CS
    turn=0;
    RS;
}
```

```
while(true)
{
    while(turn);
    turn lock=1;
    //CS
    turn lock=0;
    RS;
}
```

## Question 2

*lock=False;*

```
while(true)  
{  
    while(lock!=False);  
    CS  
    lock=True;  
    RS;  
}
```

```
while(true)  
{  
    while(lock!=True);  
    CS  
    lock=False;  
    RS;  
}
```

## Question 3

```
lock=False;
```

```
while(true)
```

```
{
```

```
    while(lock ==False)
```

```
    {
```

```
        lock = True;
```

```
    }
```

```
    CS
```

```
    lock=False;
```

```
    RS;
```

```
}
```



## Question 4

*Boolean lock= True;*

```
while(true)  
{  
    while(lock)  
    {  
        CS  
        lock = False  
    }  
    lock=True;  
    RS;  
}
```



## Topic : TestAndSet()



```
Boolean Lock=False;
```

```
boolean TestAndSet(Boolean *trg){  
    boolean rv = *trg;  
    *trg = True;  
    Return rv;  
}
```

```
while(true)
```

```
{  
    while(TestAndSet(&Lock));  
        CS  
    Lock=False;  
}
```



## Topic : Swap()



Boolean Key; //Local

Boolean Lock=False; //Shared

```
void Swap(Boolean *a, Boolean *b)
```

```
{
```

```
    boolean temp = *a;
```

```
    *a=*b;
```

```
    *b=temp;
```

```
}
```

```
while(true){
```

```
    Key = True;
```

```
    while (key==True)
```

```
        Swap(&Lock, &Key);
```

```
        CS
```

```
        Lock=False;
```

```
}
```



## Topic : Critical Section Solution

2 binary semaphores  $S_1 = \overset{0}{\cancel{1}}$ ,  $S_2 = \overset{1}{\cancel{0}}$

|       |               |            |  |            |       |
|-------|---------------|------------|--|------------|-------|
| $P_1$ | $\rightarrow$ | wait(S1)   |  | wait(S2)   | $P_2$ |
|       |               | wait(S2)   |  | wait(S1)   |       |
|       |               | C.S.       |  | C.S.       |       |
|       |               | signal(S1) |  | signal(S2) |       |
|       |               | signal(S2) |  | signal(S1) |       |

$P_1, P_2$  are in livelock  
 $\Downarrow$   
deadlock





## 2 mins Summary

**Topic**

**Banker's Algorithm**

**Topic**

**Deadlock Detection**



**Happy Learning**

**THANK - YOU**

